Bacharelado em Ciência da Computação

Fase 2

A AURORA AJUSTA A TRAJETÓRIA NA APROXIMAÇÃO A MARTE

MGPEB: Módulo de Gerenciamento de Pouso e Estabilização de Base

Repositório no GitHub:
github.com/FIAP-CC2026/FIAP-Ciencia-da-computacao-fase-2

TRABALHO INTEGRADO

Grupo 27

Gabriel Ferreira da Silva
José Kauan Medeiros Machado
Murilo Ribeiro Falconeri
Gustavo Rocha Caxias

2026

1. Introdução

O MGPEB organiza a sequência de pousos da missão Aurora Siger. O protótipo integra cadastro de módulos, estruturas lineares, busca, ordenação, regras booleanas, modelagem matemática, contexto computacional e princípios de ESG para apoiar decisões de pouso em Marte.

2. Cenário e módulos do sistema

A base inicial recebe seis módulos especializados: HAB-01, habitação; ENE-01, energia; LAB-01, laboratório; LOG-01, logística; MED-01, suporte médico; e COM-01, comunicações. Cada registro contém identificação, tipo, prioridade, combustível, massa, criticidade da carga, horário de chegada, integridade dos sensores e número de tentativas.

A fila pode ser organizada por chegada, menor combustível ou maior prioridade. Nas prioridades iguais, o sistema considera menor combustível e chegada mais cedo. A simulação avalia um módulo por ciclo, de acordo com as condições de atmosfera e área de pouso.

Os módulos representam necessidades diferentes da base: HAB-01 fornece habitação e suporte à vida; ENE-01 leva equipamentos de geração e armazenamento de energia; LAB-01 concentra instrumentos científicos; LOG-01 transporta ferramentas e suprimentos; MED-01 reúne recursos médicos; e COM-01 permite a comunicação da base. A prioridade varia de 1 a 5, sendo 5 a maior. Combustível é registrado em percentual, massa em quilogramas e a criticidade indica o impacto operacional da perda da carga.

Além dos atributos estáticos, o protótipo acompanha condições que mudam durante a execução. Combustível e tentativas podem ser atualizados quando ocorre ESPERA. Os sensores de COM-01 são cadastrados com falha para exercitar o caminho de alerta, enquanto ENE-01 possui combustível abaixo do mínimo de 30%. O cenário, portanto, contém exemplos que permitem observar autorização, espera e alerta sem depender de dados externos.

3. Estruturas de dados, busca e ordenação

Os módulos são dicionários armazenados em listas. A fila de pouso usa pop(0) para atender o primeiro item e append() para recolocar um módulo adiado no final, caracterizando atendimento FIFO após a ordenação inicial.

As listas pousados, em_espera e em_alerta registram os resultados da simulação. A lista historico atua como pilha: cada decisão é inserida com append() e a exibição das últimas decisões usa pop(), seguindo LIFO.

O programa usa busca linear para combustível, prioridade, tipo e criticidade. A busca binária localiza o horário de chegada em uma lista ordenada. Bubble sort, selection sort e insertion sort demonstram três critérios de organização da fila e são adequados à quantidade reduzida de módulos prevista no cenário.

Os dicionários mantêm os atributos de cada módulo no mesmo registro, o que permite atualizar combustível e tentativas sem perder identificação, prioridade, massa, criticidade e horário de chegada. As listas pousados, em_espera e em_alerta registram destinos distintos. Quando um módulo recebe ESPERA, ele continua acompanhado em em_espera e retorna ao final da fila; ao ser avaliado novamente, sai dessa lista antes de receber uma nova decisão. Ao término da simulação, todos os módulos foram classificados como pousados ou em alerta.

A fila usa pop(0) para remover o próximo módulo e append() para recolocá-lo no final quando necessário. Esse comportamento ilustra FIFO: depois da ordenação inicial, quem está no início é atendido primeiro. Em uma lista Python, remover o primeiro elemento tem custo O(n), pois os demais itens são deslocados. Para os seis módulos do cenário, essa escolha é adequada e deixa o mecanismo de fila visível para fins didáticos.

As buscas por combustível, prioridade, tipo e criticidade são lineares, com custo O(n). A busca binária pelo horário exige que a lista esteja ordenada e reduz a procura para O(log n). As ordenações bubble sort, selection sort e insertion sort têm custo $O(n^2)$ no pior caso, mas permitem comparar critérios diferentes e são compatíveis com o conjunto pequeno de dados. As cópias usadas nas ordenações preservam a lista original, embora os dicionários internos continuem representando os mesmos módulos.

4. Regras lógicas e diagramas de portas

4.1 Entradas e decisão

O circuito usa quatro entradas booleanas: C, combustível de pelo menos 30%; S, sensores íntegros; A, vento de até 60 km/h sem tempestade; e L, área de pouso livre. As expressões são:

AUTORIZADO = C AND S AND A AND L
ALERTA = NOT (C AND S)
ESPERA = (C AND S) AND NOT (A AND L)

Assim, problemas de combustível ou sensores geram alerta. Quando o módulo está apto, mas a atmosfera ou a área não permitem a operação, o resultado é espera.

4.2 Validação visual

O circuito foi testado em três situações representativas: todos os sinais adequados, condições ambientais inadequadas e falha no módulo.

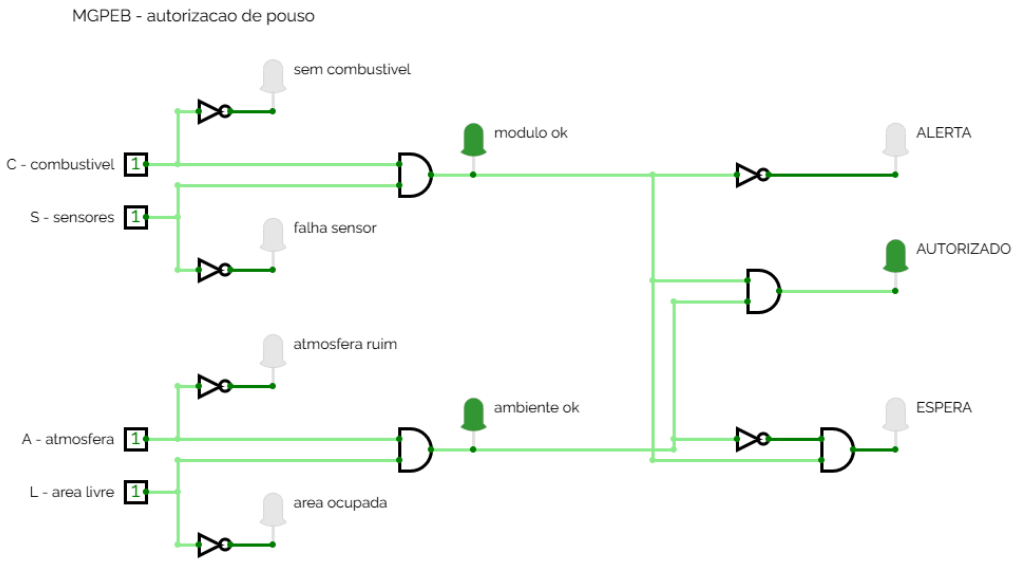


Figura 1. Circuito com pouso autorizado.

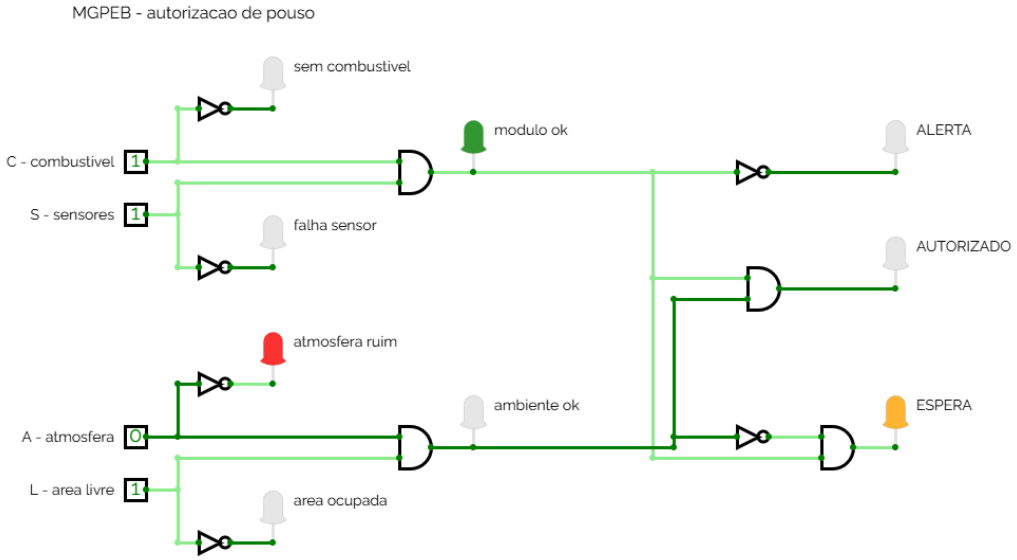


Figura 2. Circuito com pouso em espera.

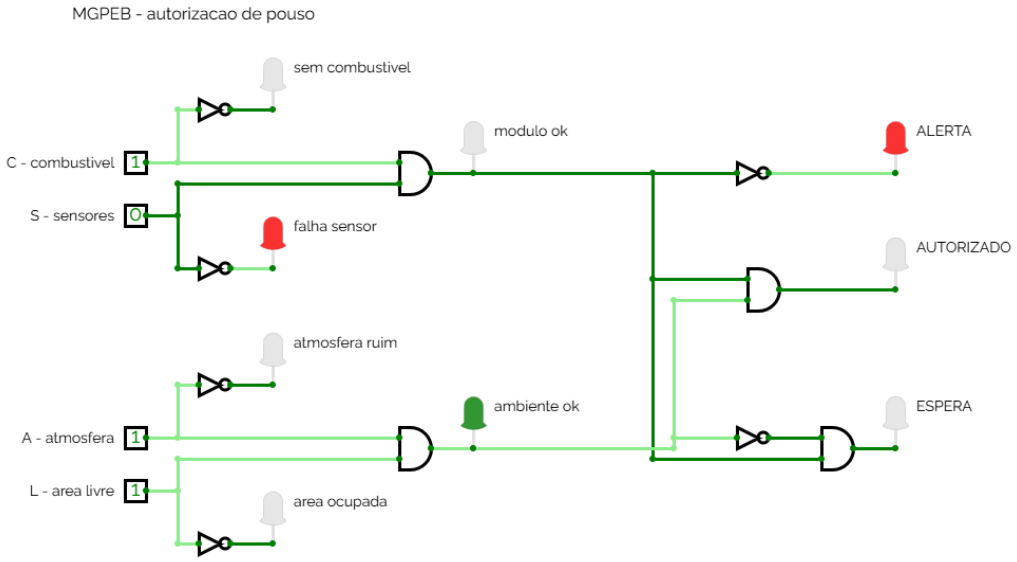


Figura 3. Circuito com alerta.

A validação não depende apenas de um exemplo favorável. Com quatro entradas booleanas, existem 16 combinações possíveis. Quando C, S, A e L valem 1, somente AUTORIZADO deve ser ativado. Quando C e S valem 1, mas A ou L valem 0, o resultado deve ser ESPERA. Quando C ou S vale 0, o resultado deve ser ALERTA, independentemente das condições ambientais. Essas relações são reproduzidas pela função autorizar_pouso e permitem verificar a coerência entre circuito e programa.

Os diagramas apresentados mostram três cenários representativos. O primeiro reúne sinais adequados e confirma a autorização. O segundo mantém combustível e sensores corretos, mas altera uma condição ambiental, produzindo espera. O terceiro representa falha de módulo e produz alerta. Os LEDs associados às condições ajudam a identificar se o impedimento vem de combustível, sensores, atmosfera ou ocupação da área.

5. Protótipo em Python

O programa é executado com python3 main.py e não usa dependências externas. O arquivo main.py coordena cadastro, consultas, ordenação e simulação. O pacote mgpeb separa entrada, exibição, módulos, buscas, ordenações, regras de pouso e ciclos de simulação.

A simulação chama autorizar_pouso, a mesma regra apresentada no circuito. Ela avalia combustível, sensores, vento, tempestade e área livre para retornar AUTORIZADO, ALERTA ou ESPERA. Em caso de espera, o módulo perde 5% de combustível e retorna ao final da fila.

O fluxo é dividido em responsabilidades para tornar a leitura e os testes mais simples. Em entrada.py ficam o cadastro e a validação dos dados informados pelo usuário; modulos.py concentra a criação dos registros; buscas.py e ordenacao.py implementam os algoritmos estudados; regras_pouso.py contém a decisão lógica; e simulacao.py executa os ciclos, atualiza a fila e registra o histórico. Essa organização evita concentrar todo o comportamento em um único arquivo.

Durante a execução, o usuário pode usar os seis módulos cadastrados ou inserir novos módulos. O programa demonstra buscas por menor combustível, maior prioridade, tipo e criticidade, além de busca binária pelo horário de chegada em uma lista previamente ordenada. Depois, a fila é organizada por chegada, combustível ou prioridade. Na ordenação por prioridade, os empates são resolvidos primeiro pelo menor combustível e depois pelo horário mais cedo, tornando o critério explícito.

A simulação também evidencia a integração entre software e lógica digital. No exemplo fornecido, ENE-01, com 28% de combustível, recebe ALERTA; HAB-01 encontra vento acima do limite, entra em ESPERA, perde 5% de combustível e

retorna ao fim da fila; em um ciclo posterior, com ambiente adequado, é autorizado. LAB-01, com condições favoráveis, é autorizado. Os três resultados demonstram que a função em Python segue a mesma regra apresentada pelos diagramas.

6. Modelagem matemática

O modelo estima o combustível gasto em função da velocidade de descida. Considerando uma fase final de altura H, gravidade marciana g, massa m e velocidade de saída dos gases v_e, o esforço do motor é descrito por:

$$\Delta v(v) = v + gH / v$$
$$F(v) = (m / v_e) \cdot \Delta v(v)$$

A primeira parcela representa a frenagem e a segunda representa a perda por gravidade durante a descida. Para o exemplo de um módulo de 12.000 kg, H = 1.000 m, g = 3,71 m/s² e v_e = 3.000 m/s, o menor consumo ocorre próximo de 61 m/s, com cerca de 487 kg de combustível.

No gráfico de F(v), a curva diminui até um ponto mínimo e volta a crescer: descer devagar demais aumenta o gasto por gravidade, enquanto descer rápido aumenta o custo de frenagem. O MGPEB pode usar essa estimativa para priorizar módulos com pouca reserva e estimar o tempo de ocupação da área de pouso. Trata-se de um modelo didático, com valores ilustrativos e sem representar integralmente uma missão real.

O termo gH/v representa a perda por gravidade: quanto menor a velocidade de descida, maior o tempo com o motor compensando a atração gravitacional. Já o termo v representa o esforço de frenagem. A função reúne esses dois efeitos e explica por que não basta escolher a menor velocidade possível. O resultado não pretende reproduzir todos os fatores de uma missão real, como atmosfera, variação de massa e controle de atitude, mas oferece uma aproximação útil para comparar escolhas.

Com os parâmetros adotados, a velocidade de menor consumo é obtida quando os dois termos de Δv se equilibram, isto é, em $v = \sqrt{(gH)}$. Para g = 3,71 m/s² e H = 1.000 m, o valor é aproximadamente 61 m/s. Nessa condição, o consumo estimado é cerca de 487 kg. Em uma descida a 10 m/s, a parcela de gravidade cresce e o

consumo estimado sobe para aproximadamente 1.524 kg, evidenciando a importância de evitar descidas excessivamente lentas.

A análise também mostra como os parâmetros influenciam a decisão. Aumentar a altura H eleva a velocidade ótima; aumentar a massa m eleva a quantidade de combustível necessária, mas não altera essa velocidade no modelo; e uma maior velocidade de saída dos gases reduz a massa de combustível estimada. No MGPEB, a função pode apoiar a comparação entre reservas de combustível, a priorização de módulos vulneráveis e a estimativa do tempo de ocupação da área de pouso.

7. História da computação e limitações de hardware

A evolução que vai dos primeiros computadores eletrônicos aos sistemas embarcados tornou viável automatizar cálculos e decisões de missão. Em Marte, limitações de energia, memória, processamento, radiação e atraso de comunicação favorecem estruturas simples, algoritmos previsíveis e decisões autônomas.

No MGPEB, listas e algoritmos clássicos são adequados ao conjunto pequeno de módulos e facilitam testes. A redundância de sensores e um registro persistente de decisões são melhorias futuras importantes para lidar com falhas e permitir auditoria.

A relação entre sustentabilidade e computação aparece nas decisões de projeto. Estruturas pequenas reduzem uso de memória; algoritmos simples facilitam testes; e regras booleanas explícitas tornam as decisões verificáveis. Em sistemas embarcados, previsibilidade é tão importante quanto desempenho, pois falhas de processamento, energia ou comunicação podem impedir uma intervenção externa imediata.

Os primeiros computadores eletrônicos mostraram a possibilidade de automatizar cálculos extensos. A evolução dos transistores, circuitos integrados e computadores embarcados tornou possível levar processamento a equipamentos móveis e remotos. Para uma operação em Marte, a escolha por listas, busca linear e ordenações clássicas se justifica pelo conjunto reduzido de módulos e pela

facilidade de inspeção do comportamento do programa. A complexidade menor ajuda a limitar riscos e torna o protótipo adequado ao objetivo didático.

8. ESG e governança

A seleção da área de pouso deve evitar regiões ambientalmente sensíveis, preservar áreas de interesse científico e reduzir riscos à base. A gestão de combustível, energia, resíduos e recursos locais precisa usar limites de consumo e critérios transparentes de prioridade.

A governança deve registrar decisões, permitir revisão das regras e justificar exceções, especialmente em situações excepcionais. Como a comunicação com a Terra tem atraso, o sistema precisa decidir localmente, mas suas decisões devem permanecer auditáveis. Dessa forma, a ordem da fila e o uso de recursos deixam de ser apenas escolhas técnicas e passam a considerar impactos ambientais, sociais e de governança.

A dimensão social envolve proteger a equipe e distribuir recursos escassos de forma justificável. A prioridade operacional não substitui a verificação das condições mínimas de segurança: um módulo com combustível insuficiente ou sensores em falha é encaminhado para alerta, enquanto um módulo íntegro espera quando o impedimento é ambiental. Essa separação evita que a urgência seja usada como justificativa para pousos em condições inseguras.

A governança proposta depende de critérios claros, registro das decisões e possibilidade de revisão. Cada ciclo guarda identificação do módulo, condição observada e resultado da regra. Em uma evolução do protótipo, esse histórico deve ser persistido, sem ser consumido na exibição, permitindo auditoria posterior. Indicadores como combustível gasto, quantidade de esperas, alertas por sensores e ocupação da área ajudam a avaliar se a operação mantém coerência com os limites definidos pela equipe.

9. Considerações finais

O MGPEB demonstra a integração entre lógica digital, programação em Python, estruturas lineares, busca, ordenação, modelagem matemática, história da computação e ESG. O projeto organiza seis módulos, simula ciclos de decisão e mostra como a disponibilidade de recursos interfere na prioridade de pouso.

Como evolução, o sistema pode manter o histórico de decisões de forma persistente e ampliar os critérios de segurança. A integração atual entre circuito e simulação torna o protótipo coerente, verificável e adequado ao cenário proposto.Os resultados da simulação confirmam a aplicação consistente da regra de decisão. A autorização ocorre apenas quando combustível, sensores, atmosfera e área livre estão adequados. A espera preserva a regra de segurança quando o problema é ambiental e ainda demonstra o impacto operacional da perda de combustível e do retorno ao final da fila. O alerta, por sua vez, impede que um módulo com falha própria seja tratado como apto para pousar.

O protótipo também evidencia limites que podem orientar uma próxima versão. O modelo matemático funciona como apoio à análise, mas não controla automaticamente a simulação; o histórico é mantido apenas durante a execução; e critérios adicionais podem ampliar a avaliação de risco. Essas limitações não comprometem a proposta didática, pois tornam claros os pontos de evolução e mostram como lógica, programação, matemática e governança podem ser integradas em uma mesma decisão operacional.